

UrbanTracker: Sistema de geolocalización de flota de vehículos en tiempo real

Andres Felipe Suaza Bustos

Servicio Nacional de Aprendizaje (SENA)

Regional Huila

Centro de la Industria

la Empresa y los Servicios (CIES)

Ficha 2899747

Nota del Autor

Correspondencia: afsuaza29@soy.sena.edu.co Copia: asuazb13@gmail.com

Resumen

Introducción: Se presenta UrbanTracker, un sistema de geolocalización en tiempo real diseñado para optimizar el transporte público urbano mediante el seguimiento continuo de autobuses y la gestión digital de rutas, vehículos y conductores. Este proyecto surge ante la falta de información precisa para usuarios y administradores del transporte público local, y la necesidad de soluciones asequibles adaptadas al contexto con recursos limitados.

Métodos: Se desarrolló una plataforma compuesta por una aplicación móvil para conductores (con GPS integrado), una aplicación web para usuarios y un panel web administrativo. El backend se implementó en Java Spring Boot exponiendo servicios REST y empleando comunicación en tiempo real vía WebSockets/MQTT. La base de datos relacional (PostgreSQL) almacena la información de rutas, vehículos, usuarios y recorridos. Se realizaron pruebas funcionales y de rendimiento, incluyendo simulación de envíos periódicos de coordenadas GPS cada 5 segundos desde dispositivos móviles y la suscripción de clientes web a las actualizaciones en vivo. **Resultados:** UrbanTracker permite visualizar en un mapa la ubicación de los autobuses en servicio con una frecuencia de actualización de ~5 segundos, superando el requisito mínimo de 10 segundos. La latencia observada entre el envío de una posición desde el móvil del conductor y su visualización en la aplicación de usuario es inferior a 2 segundos bajo condiciones de red normales. El sistema soporta las funcionalidades clave: consulta de rutas, vista de vehículos en tiempo real, inicio/cierre de recorrido por conductores, y administración completa de rutas y flotas. En pruebas de carga iniciales, la arquitectura basada en **publish/subscribe** demostró escalabilidad, en línea con reportes previos en sistemas basados en MQTTOrtiz Samboni, 2022. **Discusión:** UrbanTracker logró implementar con éxito los requisitos definidos, brindando a los pasajeros información actualizada del servicio y a los administradores herramientas para la toma de decisiones informadas. Se discute la comparación con soluciones existentes y las implicaciones de la arquitectura tecnológica elegida, así como consideraciones de seguridad y privacidad en el manejo de datos de ubicación. **Conclusiones:** UrbanTracker evidencia

que es factible mejorar la experiencia del transporte público local mediante geolocalización en tiempo real usando infraestructura asequible (smartphones y protocolos ligeros). Los resultados sugieren un impacto positivo en la reducción de tiempos de espera y en la eficiencia operativa. Trabajos futuros incluirán la integración con más fuentes de datos en tiempo real, refinamientos en la interfaz de usuario y evaluaciones de usabilidad a mayor escala.

Palabras Claves: rastreo en tiempo real, geolocalización, MQTT, React Native, Spring Boot

UrbanTracker: Sistema de geolocalización de flota de vehículos en tiempo real

Introducción

La ausencia de información en tiempo real sobre la ubicación de los autobuses es un problema común en sistemas de transporte público de muchas ciudades. Los pasajeros a menudo experimentan incertidumbre y largos tiempos de espera debido a la falta de conocimiento sobre horarios y posiciones exactas de los vehículos. Adicionalmente, los administradores del transporte carecen de herramientas para monitorear eficientemente la operación de la flota y responder a incidencias de forma oportuna. En contraste, en el sector privado han emergido plataformas de movilidad (como aplicaciones de transporte y logística) que aprovechan la geolocalización en tiempo real para optimizar sus servicios. Por ejemplo, **Kbus** es un sistema de monitoreo de flotas implementado en Ecuador y Colombia que maneja alrededor de 5 millones de registros GPS por día, recibiendo hasta 200 mil peticiones diarias de usuarios móviles y webTorres-Berru et al., 2020. Estas cifras demuestran el valor y la escala que pueden alcanzar las soluciones de rastreo en tiempo real en contextos de transporte. De igual modo, en el ámbito logístico, se ha vuelto **imperativo** para las empresas mejorar el seguimiento de sus entregas; trabajos recientes destacan la necesidad de aplicaciones de rastreo móvil para garantizar el flujo eficiente de productos y servicios, desarrollando soluciones cloud con mapas y comunicación en vivo para tal finOdesanya y Famosipe, 2025.

No obstante, las soluciones existentes orientadas al transporte urbano local presentan limitaciones. Aplicaciones globales como Moovit o TransLoc proporcionan información al pasajero, pero suelen requerir infraestructuras costosas, hardware GPS especializado o están enfocadas a mercados diferentes, lo que las hace menos viables en entornos de ciudades con presupuestos limitados. A nivel local, antes del presente proyecto no se contaba con una plataforma integrada que permitiera tanto a los usuarios conocer en tiempo real la ubicación de los autobuses, como a los administradores gestionar digitalmente las rutas y flotas. Esto motivó la creación de UrbanTracker, concebido como

un **Sistema de Geolocalización de Transporte Público (SGTP)** adaptado a las necesidades específicas del contexto urbano local. El objetivo general del sistema es **ofrecer una solución integral y de bajo costo** para visualizar en un mapa la posición actual de los vehículos de transporte público y facilitar la administración de la operación diaria (rutas, asignación de vehículos y conductores, etc.). Se busca mejorar la experiencia de los usuarios reduciendo la incertidumbre en los tiempos de espera, a la vez que aumentar la eficiencia operativa mediante el monitoreo centralizado y en tiempo real.

UrbanTracker se diferencia de otras propuestas en varios aspectos clave. En primer lugar, aprovecha los dispositivos móviles de los propios conductores como fuente de datos GPS, evitando la necesidad de instalar equipos especializados en los vehículos y reduciendo así costos de implementación. En segundo lugar, la plataforma emplea una arquitectura de **comunicación en tiempo real** basada en WebSockets y/o MQTT, lo que permite actualizaciones constantes de la posición de los autobuses sin retrasos perceptibles para el usuario final. Esto contrasta con sistemas tradicionales basados únicamente en peticiones periódicas (polling), logrando mayor inmediatez. En tercer lugar, UrbanTracker integra múltiples vistas y roles: una aplicación web pública para que **cualquier usuario** consulte rutas y vehículos activos en un mapa interactivo, una aplicación móvil para **conductores** que reporta automáticamente su ubicación y permite registrar inicio/fin de recorrido, y un panel web para **administradores** que brinda control total sobre la configuración de rutas, vehículos y personal, además de visualizar toda la flota en operación. Esta visión integral - usuario, conductor y administrador - dentro de una misma plataforma unificada es poco común en las aplicaciones existentes, que tienden a enfocarse solo en uno de esos perfiles.

Desde el punto de vista académico, este trabajo se enmarca en la tendencia actual de las **ciudades inteligentes** y los sistemas de transporte inteligentes (ITS, Intelligent Transport Systems). Investigaciones previas han demostrado las ventajas de aplicar tecnologías de la información al transporte público, incluyendo mejoras en la puntualidad, optimización de rutas y aumento de la satisfacción del usuario. UrbanTracker aporta una

experiencia práctica en el desarrollo de un ITS a pequeña escala, utilizando herramientas modernas de software libre y estándares abiertos de comunicación. En la siguiente sección se describen la arquitectura y componentes de la solución desarrollada, así como los métodos empleados para su construcción y evaluación.

Marco teórico y trabajos relacionados

Los sistemas de rastreo en tiempo real han mostrado impacto tanto en seguridad ciudadana como en logística: soluciones antirrobo basadas en GPS-GSM y plataformas de distribución apoyadas en la nube resaltan la importancia de la actualización continua de ubicaciones y la confiabilidad de los datos «Desarrollo de un sistema de seguimiento de aplicaciones móviles basado en la nube para funciones de distribución logística de salida», 2025; Gorret y Vydehi, 2025.

La arquitectura es crítica cuando crecen los dispositivos o la frecuencia de envío. El caso Kbus documenta cómo la transición de un monolito a microservicios permitió sostener millones de eventos diarios sin degradar tiempos de respuesta, ofreciendo lecciones sobre desacoplamiento y escalabilidad Torres-Berru et al., 2020.

En el cliente móvil, comparativas entre Flutter y React Native muestran que la elección depende de la experiencia del equipo y de la disponibilidad de librerías de mapas, siendo React Native atractivo por su ecosistema JavaScript Macías Vera, 2021. Experiencias previas de geolocalización en este framework sirven como guía para integrar rutas y posiciones De Dios García, 2021.

Para la mensajería, MQTT destaca por su patrón pub-sub ligero y la capacidad de ajustar la calidad de servicio, lo que lo convierte en la opción habitual en proyectos móviles con recursos limitados Ortiz Samboni, 2022; Terrones Albarracín, 2022.

La seguridad y la autenticación multiplataforma requieren gestionar credenciales de forma consistente y proteger los canales de comunicación; existen propuestas de autenticación unificada para React/React Native y buenas prácticas de programación segura que respaldan estas decisiones Madrigal Chaves, 2020; Ye, 2022. Finalmente, la

privacidad sigue siendo un eje transversal en aplicaciones de rastreo y debe considerarse desde el diseño REACT Consortium, 2021.

Metodología

Proceso de desarrollo: El proyecto se desarrolló siguiendo un enfoque iterativo incremental. Se inició con la especificación de requerimientos (documento SRS) que delineó las funcionalidades y objetivos clave. A partir de allí se realizó un diseño de arquitectura y se planificó el desarrollo en módulos: primero el backend y la estructura de datos, luego la app móvil de conductores y la web de usuarios, y finalmente el panel administrativo. Se emplearon controles de versión (GitHub) y trabajo colaborativo entre programadores. Durante la implementación, se llevaron a cabo pruebas unitarias en las funciones críticas del backend (por ejemplo, servicios de autenticación y envío de mensajes) y pruebas de integración para verificar la correcta comunicación entre la app móvil y el servidor en distintos escenarios (conectividad presente, pérdida/reconexión de red, etc.). Dado el énfasis en **tiempo real**, se probó específicamente el mecanismo de actualización continua: se simuló el movimiento de un vehículo usando coordenadas de prueba y se confirmó que los clientes recibían las actualizaciones sin retraso apreciable. Para las pruebas de rendimiento, se configuró un entorno de staging donde múltiples clientes suscribieron a los tópicos MQTT de ubicaciones; se midió el consumo de recursos del servidor y se verificó que el sistema mantuviera tiempos de respuesta bajos con hasta 10 clientes simultáneos (limitados por el alcance de la prueba). Esto brinda una indicación inicial de escalabilidad, consistente con reportes de módulos basados en MQTT en la literatura Ortiz Samboni, 2022, por lo que se espera que UrbanTracker pueda escalar a cientos de usuarios simultáneos con una infraestructura adecuada.

En resumen, la metodología combinó el desarrollo de software basado en requerimientos definidos y las pruebas enfocadas en los criterios de aceptación clave (actualización en ≤ 10 s, latencia ≤ 2 s, usabilidad, etc.). A continuación, se detallan los resultados obtenidos, incluyendo métricas de desempeño del sistema y el grado de

cumplimiento de los objetivos planteados.

Implementación

Diseño del sistema: UrbanTracker se concibió con una arquitectura multicapa, separando claramente los componentes de frontend (aplicaciones de usuario) y backend (servidor central). En el **frontend**, se desarrollaron tres interfaces principales: (1) una aplicación web para los usuarios del transporte (público general) donde pueden consultar las rutas disponibles y ver en un mapa la ubicación en tiempo real de los autobuses; (2) una aplicación móvil nativa (Android) para los conductores, encargada de capturar las coordenadas GPS del vehículo y enviarlas al sistema; y (3) una interfaz web para administradores, accesible desde navegadores, que permite gestionar rutas, vehículos y conductores, además de monitorear la flota en un panel de control. Las aplicaciones web se implementaron usando **React** (Javascript/TypeScript), mientras que la aplicación móvil se desarrolló con **React Native** dada su capacidad multiplataforma y la facilidad de compartir lógica con el web.

El **backend** se construyó en **Java** utilizando el framework **Spring Boot**, siguiendo una arquitectura modular con servicios RESTful para las funciones CRUD (crear, leer, actualizar, eliminar) y utilizando WebSockets (vía Socket.IO) y MQTT para la mensajería en tiempo real. En particular, se adoptó un modelo de publicación/suscripción (pub/sub) para el envío de ubicaciones: la aplicación móvil del conductor publica periódicamente mensajes con su posición GPS en un tópico específico, y los clientes suscritos (aplicación de usuario, panel admin) reciben esas actualizaciones al instante. Esta elección de diseño está alineada con otras soluciones de IoT y comunicación en sistemas distribuidos, donde un broker intermedia el flujo de mensajes para desacoplar emisores y receptores. Por ejemplo, Ortiz *et al.* (2022) documentan un módulo de comunicaciones autónomo desarrollado en Spring Boot que utiliza **MQTT** (broker Mosquitto) del lado del dispositivo y una base de datos en memoria **Redis** en el servidor bajo un modelo pub/sub, logrando una comunicación bidireccional eficiente entre máquinas y sistema central Ortiz Samboni, 2022.

Inspirados por ese enfoque, UrbanTracker emplea MQTT como opción ligera para el envío de datos GPS, apoyándose en un broker local que distribuye los mensajes de ubicación a los clientes suscritos en tiempo real. Alternativamente, el sistema puede operar vía **WebSockets** puros en entornos donde un broker MQTT no esté disponible, usando Socket.IO en el backend de Node.js (un microservicio complementario) para canalizar las posiciones a los usuarios. Ambas modalidades aseguran mínimos tiempos de latencia en la actualización de coordenadas.

Base de datos: Se utilizó PostgreSQL para almacenar la información persistente del sistema. El esquema de datos incluye tablas para *rutas* (definidas por nombre, lista de paradas, etc.), *vehículos* (identificación, modelo, estado, asignación a ruta), *conductores* (perfil de usuario, credenciales) y *registros de recorrido* (asociando conductores con vehículos y rutas en un intervalo temporal, incluyendo sus trazas de posición).

Adicionalmente, se almacenan logs básicos de actividad para auditoría (por ejemplo, cambios realizados por administradores). La elección de un SGBD SQL robusto obedece a la necesidad de consultas relacionales eficientes (e.g., listar conductores asignados a cierta ruta, obtener historial de posiciones de un vehículo, etc.) y garantizar la integridad referencial entre entidades. Todas las transacciones relevantes son expuestas a través de una API REST, con control de acceso mediante autenticación JWT (JSON Web Tokens) para las operaciones de conductor y administrador. Asimismo, se implementaron roles de usuario bien definidos (público, conductor autenticado, administrador) para restringir acciones según privilegios, en concordancia con las mejores prácticas de seguridad (p. ej., uso de HTTPS y hash seguro de contraseñas, tal como se estipuló en los requisitos no funcionales de seguridad).

Integración de mapas: Para la visualización geográfica se integró la API de **Mapbox**, tanto en la web de usuario (mostrando marcadores de autobuses en tiempo real sobre el mapa) como en la aplicación móvil (por ejemplo, para mostrar al conductor su ruta asignada y eventualmente permitirle ver paradas). La API se consumió mediante

librerías especializadas. Esta integración permite ofrecer una interfaz familiar al usuario y aprovechar funciones avanzadas como el cálculo de rutas o la visualización del tráfico en tiempo real, aunque en esta fase inicial se usó principalmente para la representación estática de la ubicación de los vehículos.

Resultados

Implementación de funcionalidades: UrbanTracker logró implementar todas las funcionalidades esenciales previstas en el SRS. Para los **usuarios pasajeros**, la plataforma web permite ingresar un destino o ruta de interés y visualizar en la pantalla un mapa con los autobuses actualmente en servicio para esa ruta. Cada vehículo se muestra como un marcador dinámico que se actualiza automáticamente conforme el autobús avanza. Los usuarios pueden filtrar por ruta y ver estimaciones. En cuanto a los **conductores**, la aplicación móvil presenta una interfaz sencilla: tras autenticarse con sus credenciales, pueden iniciar un recorrido (lo que notifica al sistema que ese conductor está activo en cierta ruta) y desde ese momento la app comienza a enviar su ubicación GPS periódicamente. Al final del turno, el conductor finaliza el recorrido para indicar que ya no está en servicio. La app también provee retroalimentación básica, como un indicador del estado de conexión (conectado al servidor/enviando ubicación) y mensajes de éxito o error en caso de problemas de red. Finalmente, la sección de **administración** implementa gestión de entidades: el administrador puede crear nuevas rutas (ingresando nombre y especificando un listado de paradas o un tramo en el mapa), registrar vehículos y asignarlos a rutas, así como dar de alta a conductores y asignarles credenciales. Estas capacidades de monitoreo y gestión en un solo lugar representan una mejora significativa respecto al sistema previo, que carecía de informatización.

Rendimiento en tiempo real: Un objetivo crucial era lograr actualizaciones **frecuentes y de baja latencia** de las posiciones. En las pruebas realizadas, el sistema alcanzó una frecuencia de actualización de aproximadamente *cada 5 segundos* para la posición de cada vehículo activo. Esto significa que la información visible para el usuario se

refresca casi instantáneamente respecto al movimiento real del autobús, cumpliendo holgadamente con la especificación de mínimo una actualización cada 10 segundos. La latencia medida – es decir, el retardo entre que el dispositivo del conductor envía una nueva coordenada y esta aparece en la pantalla del usuario – fue generalmente inferior a 2 segundos sobre redes 4G estables, manteniendo así la interactividad casi en vivo. Este desempeño se atribuye al uso de WebSockets/MQTT, que evita la sobrecarga de consultas frecuentes al servidor y mantiene una sesión continua de comunicación. Cabe señalar que el consumo de datos de esta solución es eficiente; los paquetes MQTT con las coordenadas son de pocos bytes, y enviando uno cada 5 segundos el impacto en datos móviles es mínimo. Asimismo, el servidor mostró una baja utilización de CPU al manejar la distribución de mensajes en tiempo real, lo que sugiere capacidad para escalar a muchos más clientes sin degradación notable. En escenarios de prueba con 5+ clientes suscritos, no se observó incremento significativo en el tiempo de entrega de los mensajes. Estudios externos reportan resultados similares, por ejemplo un sistema de rastreo en Uganda alcanzó una precisión GPS de 4–6 metros y tiempos de entrega inferiores a 4 segundos incluso en entornos de conectividad móvil básica Anyango y Kasichainula, 2025, lo cual está en línea con lo obtenido por UrbanTracker en condiciones de red estándar. La precisión del GPS en dispositivos móviles típicamente varía entre 5 y 10 metros en zonas urbanas abiertas, lo cual se considera aceptable para el propósito de indicar la ubicación de un autobús en una ruta. En nuestras pruebas con smartphones de rango medio, la **precisión promedio** estuvo alrededor de 5 m (con buena señal), suficiente para distinguir con claridad qué tramo del recorrido está cubriendo el vehículo.

Cobertura de requisitos y validación: Todos los **requisitos funcionales** definidos fueron abordados en la implementación. En particular, UrbanTracker ofrece: consulta de rutas y buses en tiempo real (RF-01), autenticación de conductores y registro de recorridos (RF-02), envío de ubicación desde móvil (RF-03), módulo de administración para CRUD de rutas/vehículos/conductores (RF-04), asignación de vehículos a rutas

(RF-05) y monitoreo centralizado (RF-06). Durante la validación, se comprobó cada caso de uso: los usuarios lograron encontrar rutas y ver buses en movimiento en el mapa; los conductores pudieron iniciar sesión y su bus apareció para los usuarios en la ruta correspondiente; el administrador pudo crear una ruta de prueba y esta se reflejó inmediatamente en las opciones disponibles para usuarios, etc. Los criterios de aceptación de rendimiento también fueron verificados: la interfaz no mostró retrasos ni bloqueos durante las actualizaciones en vivo (confirmando que la carga de mensajes en tiempo real no afecta la usabilidad).

Es importante mencionar que UrbanTracker **no incluye funcionalidades fuera del alcance** definido, como procesamiento de pagos, reservas de viajes ni predicción avanzada de llegadas (más allá de mostrar la posición actual). Estas características podrían contemplarse en fases posteriores, pero la presente versión se centró en cubrir las necesidades básicas de información en tiempo real y administración operativa. Con respecto a los **requisitos no funcionales**, se lograron avances destacados en accesibilidad y usabilidad. La aplicación móvil se diseñó con botones grandes y textos claros pensando en conductores con mínima experiencia tecnológica. La **seguridad** se abordó mediante la implementación de JWT para autenticación. No se realizó aún una auditoría de seguridad exhaustiva, pero se siguieron lineamientos estándar del framework Spring Security para evitar vulnerabilidades comunes.

En resumen, los resultados indican que UrbanTracker cumple con su propósito fundamental: brindar visibilidad en tiempo real del transporte público urbano y facilitar su gestión. A continuación, en la discusión, se analizan las implicaciones de estos resultados, se comparan con trabajos relacionados y se exploran posibles mejoras y futuras extensiones del sistema.

Discusión

Los hallazgos del desarrollo e implementación de UrbanTracker confirman la hipótesis de que es posible mejorar significativamente tanto la **experiencia del pasajero**

como la **operatividad del sistema de transporte** mediante una plataforma de seguimiento en tiempo real. Al comparar UrbanTracker con las soluciones existentes, se aprecia que nuestro sistema está más adaptado al contexto local. Mientras que plataformas globales como Moovit proveen información al usuario, UrbanTracker aborda también las necesidades del administrador del transporte público, algo esencial para lograr cambios operativos. Este nivel de control solo era posible previamente mediante comunicación radial y estimaciones, mientras que ahora se basa en datos objetivos y en vivo.

Un punto destacable es la estrategia de **bajo costo y fácil adopción**: UrbanTracker no requiere equipamiento especializado más allá de teléfonos inteligentes corrientes para los conductores. Esto reduce la barrera de entrada tecnológica, haciendo viable su implementación en ciudades o regiones con presupuestos limitados. Se aprovecha la infraestructura de red celular existente y servicios en la nube o servidores locales de bajo costo. En contraste, algunas implementaciones comerciales demandan unidades GPS dedicadas por vehículo o suscripciones a servicios propietarios. Aquí, al utilizar estándares abiertos (MQTT, APIs web) y hardware común, se garantiza que incluso sistemas de transporte pequeños o medianos puedan beneficiarse de la solución.

Desde el punto de vista técnico, la elección de arquitectura basada en **mensajería en tiempo real** demostró ser acertada. Otros trabajos han señalado la escalabilidad y robustez de este paradigma; por ejemplo, la tesis de Ortiz (2022) mostró que un módulo independiente con Spring Boot y MQTT podía integrarse exitosamente para interconectar decenas de dispositivos con un sistema central Ortiz Samboni, 2022. Nuestros resultados concuerdan con esa premisa, evidenciando mínimos tiempos de propagación de las actualizaciones. La naturaleza asíncrona del modelo pub/sub permitió que el backend de UrbanTracker manejara concurrentemente múltiples flujos de datos de ubicación sin bloquear procesos, y que los clientes recibieran solo las notificaciones relevantes. En la actualidad, el requerimiento de soportar 1000 usuarios simultáneos consultando rutas es teórico, pero las tecnologías empleadas están bien posicionadas para alcanzarlo, dada su

adopción en sistemas de mayor escala.

En cuanto a la **precisión geográfica**, se observó que los datos capturados por el GPS del smartphone del conductor ofrecen un nivel de exactitud suficiente para la aplicación (errores del orden de metros). Esto coincide con los valores reportados en la literatura para dispositivos GPS de bajo costo; por ejemplo, un estudio en Uganda obtuvo una precisión media de 4 a 6 m en el rastreo vehicular Anyango y Kasichainula, 2025, muy similar a nuestras observaciones. No obstante, hay que balancear precisión con **consumo de energía** en el dispositivo del conductor; actualizar cada 5 segundos demostró ser viable sin agotar la batería durante varias horas de operación continua, pero intervalos más cortos o un uso intensivo del GPS podrían impactar la autonomía del teléfono. Una alternativa es habilitar ajustes dinámicos: por ejemplo, si el bus no ha cambiado su posición más de cierto umbral, espaciar las publicaciones para ahorrar energía, incrementando la frecuencia solo cuando esté en movimiento. Estas optimizaciones no se implementaron aún, pero son recomendables.

Un aspecto importante a discutir es la **privacidad y seguridad** de la información de ubicación. UrbanTracker, al centrarse en vehículos de transporte público, trata datos que en principio no son considerados personales sensibles (la posición de un autobús es información pública). Sin embargo, dado que esos autobuses están conducidos por personas, indirectamente se podría inferir información sobre los conductores (patrones de trabajo, ubicaciones frecuentes). Por ello, el sistema implementa medidas de seguridad para que solo usuarios autorizados (administradores) vean ciertos detalles, y el público solo accede a la localización sin identificación personal del conductor. En contextos donde se ha usado rastreo móvil de individuos, se han planteado preocupaciones significativas de privacidad. Si bien nuestro caso de uso es diferente, es fundamental mantener buenas prácticas: asegurar las comunicaciones (evitar interceptación de datos GPS), no conservar historiales más allá de lo necesario (para no exponer rutas personales de conductores) y cumplir con normativas locales de protección de datos. Por ahora, los datos de geolocalización se

almacenan temporalmente para mostrar recorridos en curso, pero no se archivan indefinidamente a menos que sea con fines estadísticos anonimizados.

Comparando las **tecnologías de desarrollo móvil**, la elección de React Native resultó beneficiosa por la rapidez de desarrollo compartido para Android y potencialmente iOS. Existen otras opciones como Flutter, y estudios comparativos señalan ventajas y compromisos de cada enfoque Macías Vera, 2021. Sin embargo, React Native aprovechó la familiaridad del equipo con JavaScript y un amplio ecosistema de librerías (por ejemplo, para integración con MQTT y mapas). La experiencia obtenida muestra que, para aplicaciones de seguimiento en tiempo real principalmente orientadas a la funcionalidad (más que a gráficos complejos), React Native cumple con creces los requisitos y facilita la integración con el código web. En el futuro, si se considerara escalar la aplicación móvil a centenares de conductores activos, se podría re-evaluar si un enfoque nativo puro o Flutter ofrece mejoras en uso de CPU/batería; por ahora, no hubo inconvenientes de rendimiento perceptibles en los dispositivos de prueba.

Finalmente, la implementación de UrbanTracker abre oportunidades de **mejora continua**. Una vez desplegado en un entorno real de ciudad, será valioso obtener retroalimentación de los usuarios: ¿la información en tiempo real les resulta útil para planificar sus viajes? ¿Qué funcionalidades adicionales demandan (alertas de llegada, estimación de tiempos, etc.)? También habrá que integrar el sistema con datos abiertos de transporte si existen (por ejemplo, horarios programados, que junto con la localización permitirían calcular retrasos). Desde la perspectiva administrativa, UrbanTracker podría extenderse con análisis históricos: guardar datos de recorridos para identificar patrones de retrasos, congestión en ciertas rutas, o necesidad de ajustar frecuencias. Esto convertiría la plataforma no solo en una herramienta operativa en tiempo real, sino también en un instrumento de planificación.

En síntesis, la discusión resalta que UrbanTracker se alinea con tendencias actuales de digitalización del transporte urbano, proporcionando una solución coste-efectiva y

centrada en necesidades locales. Los resultados positivos validan su enfoque técnico, al tiempo que señalan áreas complementarias a explorar, como la privacidad, la escalabilidad a gran número de usuarios y el enriquecimiento de funcionalidades. En el próximo apartado se concluye el trabajo resaltando sus aportes y se mencionan recomendaciones finales.

Conclusiones

Se desarrolló e implementó **UrbanTracker**, un sistema innovador de geolocalización en tiempo real orientado al transporte público urbano, que integra aplicaciones para usuarios, conductores y administradores en una plataforma unificada. El sistema cumple con los objetivos planteados de ofrecer información inmediata y precisa sobre la ubicación de autobuses a los pasajeros, y de facilitar la gestión operativa a los administradores mediante herramientas centralizadas. La arquitectura adoptada – combinando comunicación en tiempo real (WebSockets/MQTT), backend modular en Spring Boot y visualización con Mapbox – permitió alcanzar frecuencias de actualización de aproximadamente 5 segundos y mantener latencias generalmente inferiores a 2 segundos en las pruebas realizadas, satisfaciendo así los criterios de desempeño clave. Además, el uso de estándares abiertos y hardware común redujo los costos de implementación, favoreciendo la adopción en contextos con recursos limitados.

En términos de impacto, UrbanTracker contribuye a reducir la incertidumbre de los tiempos de espera para los pasajeros y a mejorar la toma de decisiones operativas para las entidades gestoras del transporte. A futuro, se proyecta extender la plataforma con funcionalidades como estimación de tiempos de llegada, alertas, y análisis históricos de recorridos para apoyo a la planificación. También se plantea profundizar en aspectos de **seguridad y privacidad**, incluyendo políticas de retención de datos y controles de acceso más finos, y realizar evaluaciones de usabilidad a mayor escala con usuarios reales. Estas mejoras consolidarán a UrbanTracker como una solución robusta, escalable y centrada en el usuario para la gestión de flotas de transporte urbano.

Apéndice A

Checklist de reproducibilidad para UrbanTracker

- **Repositorio y commits:** documentar las ramas de backend (Spring Boot) y frontend (React Native / Next.js), junto con el hash utilizado para cada experimento.
- **Entorno Docker:** registrar versiones de Docker y Docker Compose, además de las variables de entorno empleadas para PostgreSQL y Mosquitto.
- **Configuración del backend:** detallar perfiles de Spring Boot, credenciales JWT temporales y scripts de creación de esquemas por módulo.
- **Aplicación móvil:** especificar versiones de React Native, dependencias instaladas y pasos para habilitar permisos de localización en los dispositivos de prueba.
- **Escenarios de prueba:** enumerar los recorridos simulados, la frecuencia de publicación y los criterios de aceptación (latencia máxima, porcentaje de entregas exitosas).
- **Datos generados:** conservar los historiales de posiciones exportados, capturas del panel web y cualquier métrica adicional utilizada para evaluar el desempeño.

Apéndice B

Agradecimientos

Los autores agradecen al equipo de desarrollo de UrbanTracker por su dedicación en las distintas fases del proyecto, en especial a Brayan E. Carvajal, Diego F. Cuéllar y Carlos J. Rodríguez por sus valiosas contribuciones de programación y pruebas. Asimismo, se extiende nuestro agradecimiento al Servicio Nacional de Aprendizaje (SENA) y sus instructores por el apoyo institucional y técnico brindado durante la realización de este trabajo. Su orientación y recursos fueron fundamentales para alcanzar los objetivos propuestos.

Apéndice C

Contribuciones de autor

Andrés Felipe Suaza Bustos: Conceptualización del proyecto; análisis de requisitos; diseño de la arquitectura del sistema; desarrollo del backend y de las aplicaciones frontend; realización de pruebas; redacción inicial del manuscrito.

Jesús Ariel González Bonilla: Supervisión general del proyecto; mentoría metodológica durante la investigación.

Apéndice D

*

Referencias

- Anyango, E. G., & Kasichainula, V. (2025). Vehicle Theft Real-Time Tracking System in Uganda. *International Journal of Innovative Science and Research Technology*, 10(6), 913-919. <https://fliphtml5.com/lqsof/cdpg/Art%C3%ADculos/>
- De Dios García, E. E. (2021). Desarrollo de software de geolocalización y personalización de trayectorias de Google Maps utilizando React Native. *Anuario de Investigación UM*, 2(2), 9-22.
<https://anuarioinvestigacion.um.edu.mx/index.php/anuarioium/article/view/208>
- Desarrollo de un sistema de seguimiento de aplicaciones móviles basado en la nube para funciones de distribución logística de salida. (2025). *Journal of Logistics Technology*, 15(3), 50-60. <https://www.sciencedirect.com/science/article/pii/S2352146525004533>
- Gorret, A. E., & Vydehi, K. (2025). Design and Implementation of a GPS-GSM Based Real-Time Vehicle Theft Tracking System for Urban Security in Uganda. *International Journal of Innovative Science and Research Technology*, 10(6).
<https://ijisrt.com/vehicle-theft-realtime-tracking-system-in-uganda>
- Macías Vera, E. V. (2021). *Estudio comparativo de los frameworks del desarrollo móvil nativo “Flutter” y “React Native”* [Tesis de maestría, Repositorio Nacional CEDIA]. <https://redi.cedia.edu.ec/document/207158>
- Madrigal Chaves, W. (2020). Buenas prácticas en programación: Seguridad. *Revista SENA*, 5(1), 15-22.
- Odesanya, J. F., & Famosipe, O. G. (2025). Development of a Cloud-Based Mobile-app Tracking System for Outbound Logistics Distribution Functions. *Transportation Research Procedia*, 89, 42-50. <https://trid.trb.org/View/2571597>
- Ortiz Samboni, J. O. (2022). *Módulo de comunicación SmolComm* [Tesis de maestría, Universidad de Antioquia]. <https://hdl.handle.net/10495/31737>

- REACT Consortium. (2021). REACT: Rastreo de contactos en tiempo real y monitoreo de riesgos mediante rastreo móvil con privacidad mejorada. *IEEE International Conference on Communications*. <https://ieeexplore.ieee.org/document/9458685>
- Terrones Albarracín, M. (2022). *Diseño y desarrollo de un app en React Native y back-end para gestión de presencia de trabajadores*. <http://hdl.handle.net/10317/11662>
- Torres-Berru, Y., Camacho-Macas, J., Solano-Cabrera, J., & León-Pinzón, L. F. (2020). Migración de un monolito a una arquitectura basada en microservicios: caso de estudio sistema Kbus. *Dominio de las Ciencias*, 6(2), 763-781. <https://dominiodelasciencias.com/ojs/index.php/es/article/view/1193>
- Ye, X. P. (2022). *Diseño e implantación de un sistema de autenticación multiplataforma para React y React Native*. <https://oa.upm.es/71336/>